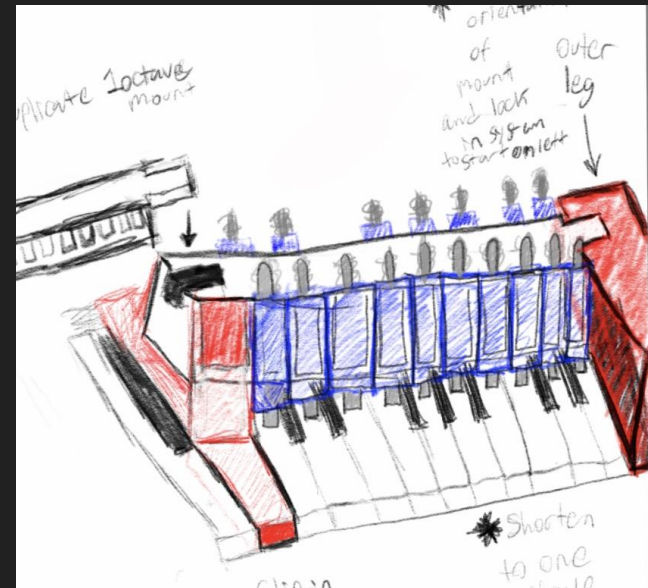


Arduino Powered Player Piano

Initial Concept:

- Create a real time midi controllable device that can trigger keys on any full-sized acoustic piano or electro-acoustic piano
- Achieve per note velocity control
- Top-mounted modular design for portability and versatility
- Use affordable parts compared to similar systems

Early sketch:



Inspiration:

Yamaha Disklavier



- \$10k-\$20k
- Internal/Non-portable system
- Restricted to a single timbre

Polyend Perc



- Limited to 3 beaters
- Not designed for keys
- Discontinued

Current Progress:

- One octave wired with Real-time midi response (through Arduino)
- 3D printed prototype modular mount
- Working breadboard circuit with room for expandability
- Chord Arpeggiator demo Max patch



Piano Demo:



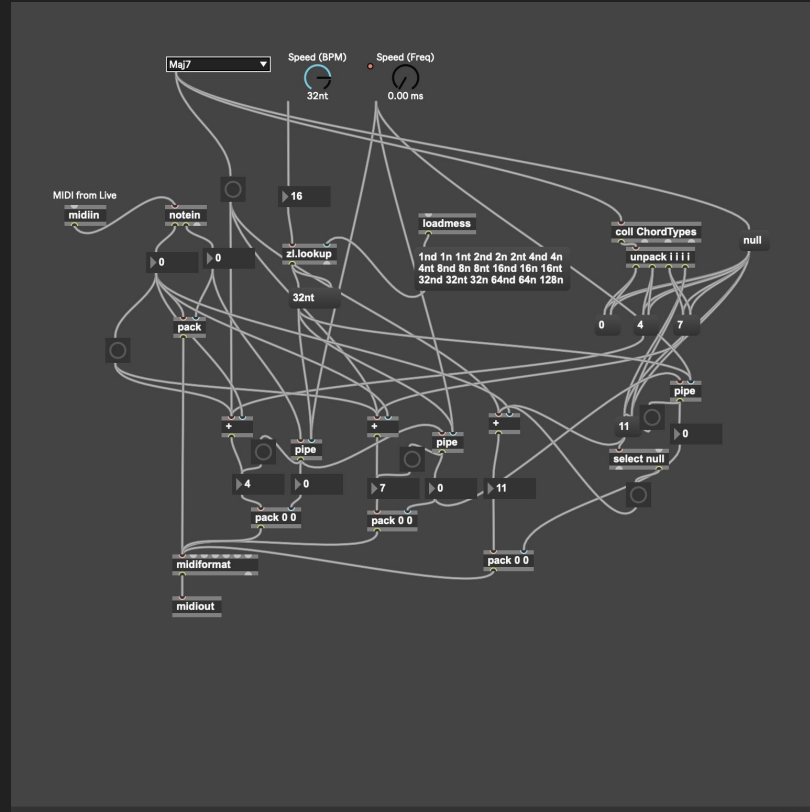
Chromatic Scale Demo:



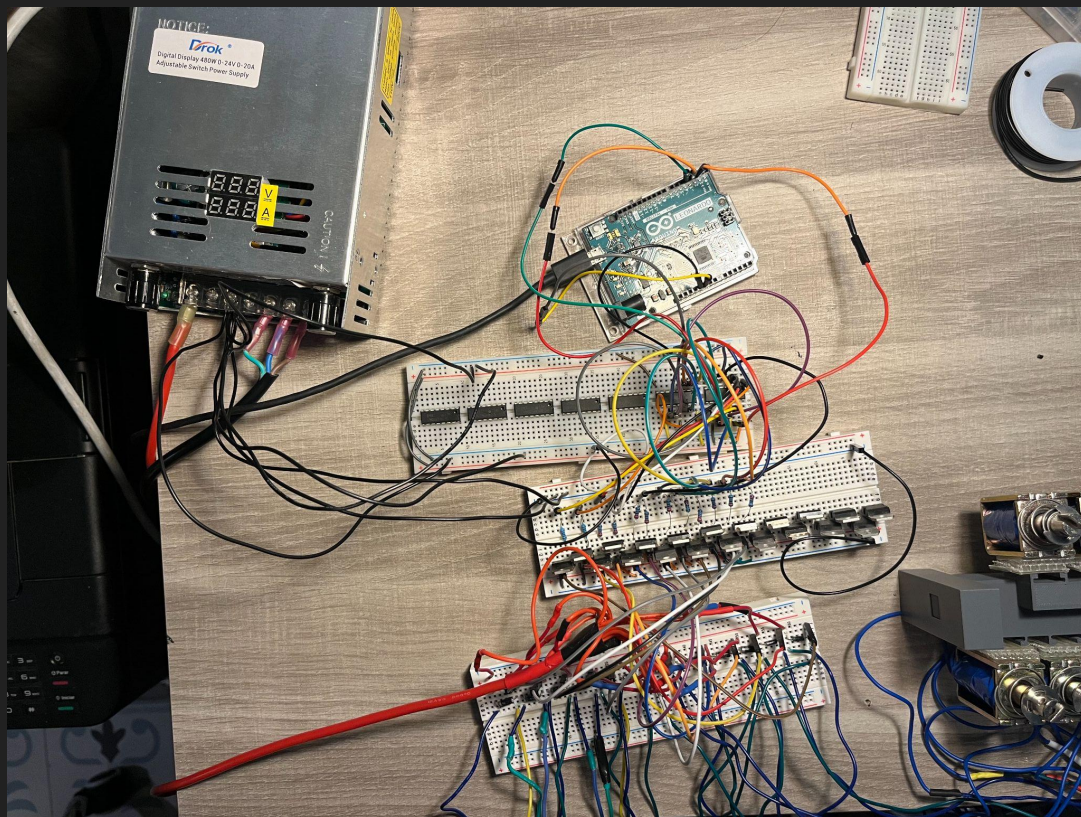
Chord Arpeggiator Demo:



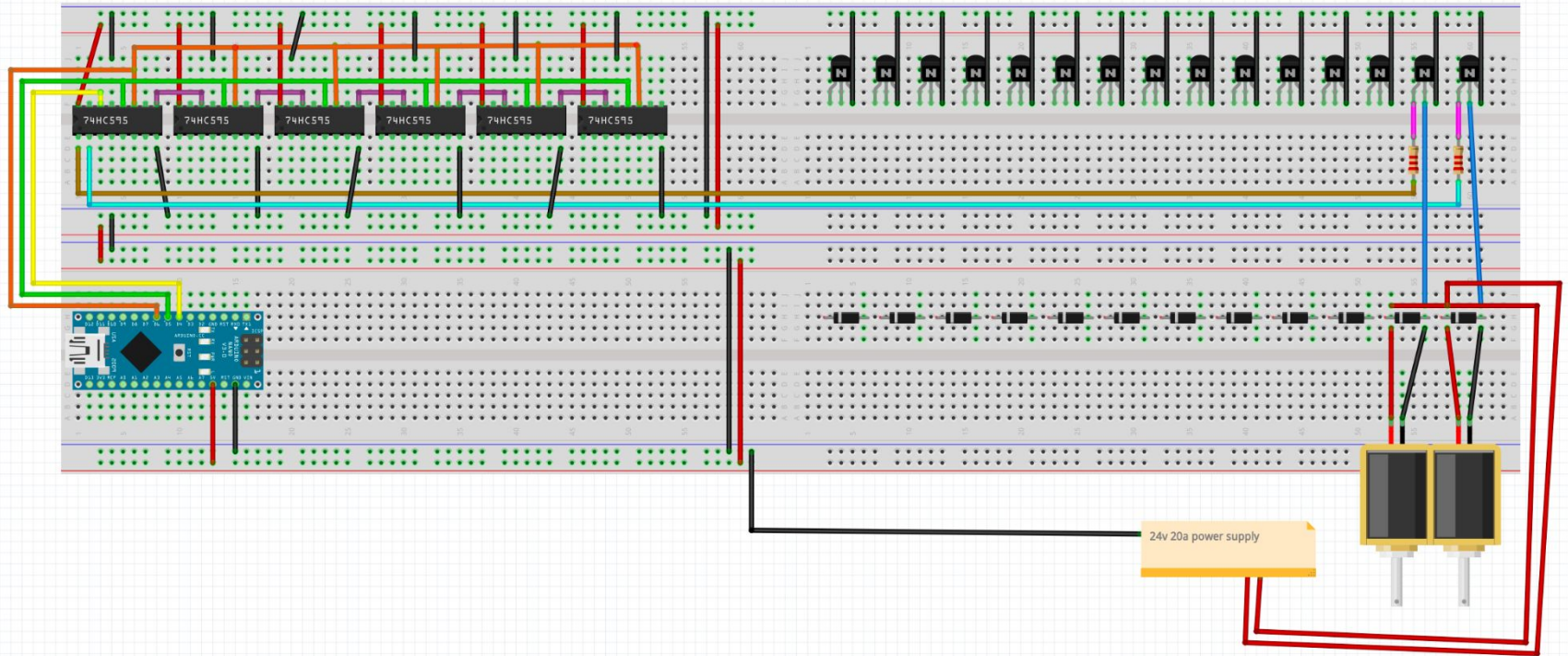
Chord Arpeggiator Max Patch:



Circuit:



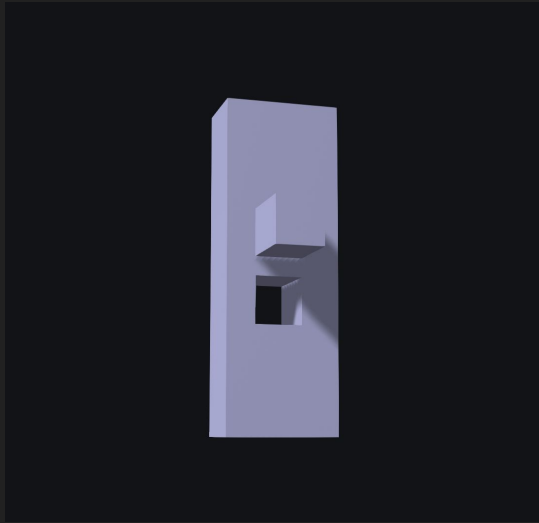
Simplified Breadboard Layout:



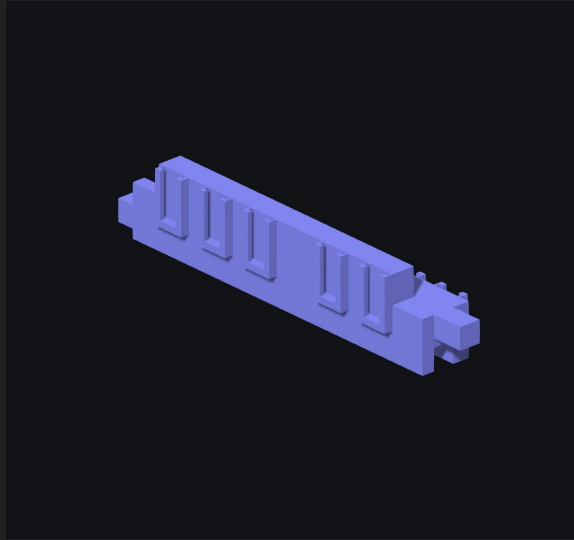
Arduino Code

```
#include <ShiftRegister74HC595.h>
#include "MIDIUSB.h"
int note = 0;
int velocity = LOW;
numberOfShiftRegisters = 2
ShiftRegister74HC595<numberOfShiftRegisters > sr(2, 3, 4);
void setup() {
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  Serial.begin(115200);
}
void noteOn(byte channel, byte pitch, byte velocity) {
  midiEventPacket_t noteOn = {0x09, 0x90 | channel, pitch, velocity};
  MidiUSB.sendMIDI(noteOn);
}
void noteOff(byte channel, byte pitch, byte velocity) {
  midiEventPacket_t noteOff = {0x08, 0x80 | channel, pitch, velocity};
  MidiUSB.sendMIDI(noteOff);
}
void midiMsg() {
  midiEventPacket_t rx;
  rx = MidiUSB.read();
  if (rx.header == 9) { //if message is a note on message
    note = (rx.byte2 - 36);
    velocity = HIGH; // (rx.byte3 * 2) - 1;
    sr.set(note, HIGH);
    Serial.print("Note On: ");
    Serial.print("Note: ");
    Serial.print(rx.byte2 - 36, DEC);
    Serial.print(" Velocity: ");
    Serial.println(rx.byte3, DEC);
  }
  else if (rx.header == 8) { //if message is a note on message
    note = (rx.byte2 - 36); //lowest key 0 mapped to FL's A2 (A0 on other DAWs)
    velocity = LOW;
    sr.set(note, LOW);
    Serial.print("Note Off: ");
    Serial.print("Note: ");
    Serial.print(rx.byte2 - 36, DEC);
    Serial.print(" Velocity: ");
    Serial.println(rx.byte3, DEC);
  }
}
void midiPlay() {
  midiMsg();
}
void loop() {
  midiMsg();
}
```

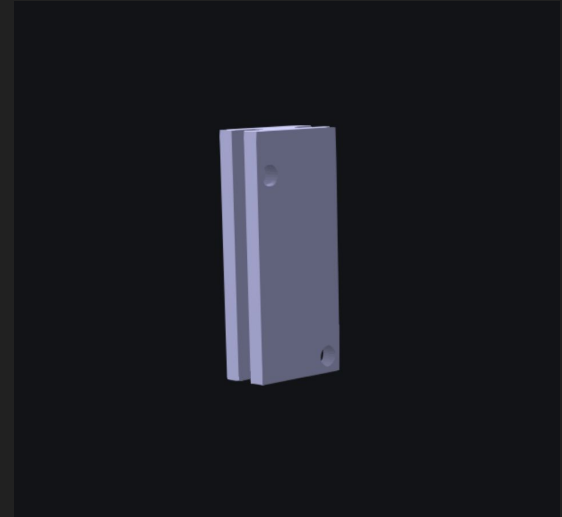
Modular Per-Octave Mount Design



<https://my.spline.design/untitled-50a336332639c8eafcf91089653de965/>



<https://my.spline.design/untitled-1add2bc49808fe7f5229e68708eabda6/>



<https://my.spline.design/untitled-7f8e0c3d5c8fdb0df727665ece6b88ea/>

Current Issues:

- Adding additional octaves of solenoids
 - Takes many hours to solder and add additional nodes of the circuit but I will attempt to get 7 octaves done by the end of the semester
- Velocity is currently broken because of Arduino Leonardo compatibility issues with ShiftRegisterPWM library
 - Tried to work around this issue by sending MIDI messages through I2C from Arduino Leonardo to Arduino Uno but was unsuccessful because of significant signal delay (500+ms) when receiving messages
 - I will try to edit the ShiftRegisterPWM library to be compatible with Arduino Leonardo or rewrite it from scratch
- Octave mounts are structurally unstable connected together
 - I will redesign the model to connect horizontally rather than vertically and reshape it as a latch
- Octave mount stands are not large or heavy enough to support the weight of the octave mounts without tipping over
 - I will redesign them to be wider and add additional metal weights for support
- Solenoid screw mounts are too thick for their holder and the screw holes are the wrong size
 - I will make the solenoid screw mount holders slightly larger and add the correct screw hole size to the piece
- Solenoids are loud
 - I am testing adding additional O-Rings to the plunger to decrease the clicking sound caused by actuation of the solenoid. If I cannot greatly improve it I may consider other ways such as an enclosure to reduce noise

Further Ideas (to be completed this semester):

Max Patches:

1. Chord Arpeggiator -

- a. Add chord extensions (9, 11, 13) as options
- b. Key selector - plays relevant chords of the selected key based on root played in midi clip

2. Physical Delay -

- a. Using midi input, plays a delayed repeat of the same notes with velocity decreasing with each delay on the physical piano instead of a delay loopback of the audio recording
- b. Parameters for delay time (midi synced or frequency), Feedback, Pitch Shift, and Mappable LFO

3. Call and Response -

- a. Use Markov Chains trained on real time midi input
- b. Plays a randomly generated midi phrase

Further:

Hardware:

- Design a PCB version of the power and arduino trigger circuit
- Redesign solenoid mounting clips on octave mount to allow for height adjustment per key
- Changeable plungers on each solenoid with variety of textures (wood, plastic, metal, felt)
- On-contact magnetic power distribution for solenoids connected to octave mount (rather than existing cables)
- Design a drum clip mount for solenoids to be able to be mounted on a drum kit
- Design a tripod mount for solenoids to be mounted on a camera tripod

Further (2):

Octave mount midi and velocity detection:

- Add proximity sensors on bottom of octave mount to be able to detect piano keys that are being played by a human (acting as a midi controller)

Hardware Live Looping:

- Audible metronome
- Record a set amount of bars to record for
- Replay user-played segment on loop using solenoids then allow additional user input via overdub

Eurorack interfacing:

- CV Input per solenoid